Control del Movimiento Robótico con ROS

Julio Garcia Ortiz

1. Entorno de trabajo 2
2. Descargamos la imagen de ROS con TurtleBot3 2
3. Creación de proyecto python 2
4. Archivo y código Python 3
5. Ejecutamos ROS 4
6. Copia del archivo Py al contenedor 4
7. Actualización de TurtleBot3 4
8. Ejecución del programa 5
9. Conclusión 5

1. Entorno de trabajo

Para comenzar la tarea nos aseguramos de que versión de docker tenemos instalada.

```
> docker --version
Docker version 29.2.1, build a5c7197
PS C:\Users\jgortiz>
```

2. Descargamos la imagen de ROS con TurtleBot3

Para trabajar con ROS necesitamos descargar la imagen desde docker, el cual incluye el entorno necesario para trabajar con TurtleBot3.

```
PS C:\Users\jgortiz> docker pull osrf/ros:noetic-desktop-full
noetic-desktop-full: Pulling from osrf/ros
13b7e930469f: Extracting [================>
```

3. Creación de proyecto python

Creamos la carpeta del proyecto y dentro el archivo de python mediante el cual controlaremos el movimiento del robot.

```
Control_Movimiento_Robot
    ros.JulioGarcía.py    U
```

4. Archivo y código Python

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import rospy
import math
from geometry_msgs.msg import Twist


def mover_recto(publisher, velocidad=0.2, distancia=1.0):
    """Mueve el robot en línea recta."""
    mensaje = Twist()
    mensaje.linear.x = velocidad
    mensaje.angular.z = 0.0
    tiempo = distancia / velocidad
    inicio = rospy.Time.now()
    rospy.loginfo(f"Moviendo recto {distancia} metros...")
    while (rospy.Time.now() - inicio).to_sec() < tiempo:
        publisher.publish(mensaje)
        rospy.sleep(0.1)
    detener_robot(publisher)
    rospy.loginfo("Movimiento recto completado.")


def girar_90_grados(publisher, velocidad_angular=0.5):
    """Gira el robot exactamente 90 grados."""
    mensaje = Twist()
    mensaje.linear.x = 0.0
    mensaje.angular.z = velocidad_angular
    angulo_objetivo = math.pi / 2
    tiempo = angulo_objetivo / velocidad_angular
    inicio = rospy.Time.now()
    rospy.loginfo("Girando 90 grados...")
    while (rospy.Time.now() - inicio).to_sec() < tiempo:
        publisher.publish(mensaje)
        rospy.sleep(0.1)
    detener_robot(publisher)
    rospy.loginfo("Giro completado.")


def detener_robot(publisher):
    """Detiene el robot enviando velocidad cero."""
    mensaje = Twist()
    publisher.publish(mensaje)
    rospy.sleep(0.5)


def main():
    """Inicializa el nodo ROS y ejecuta la secuencia de movimiento."""
    rospy.init_node('control_turtlebot', anonymous=True)
    publisher = rospy.Publisher('/cmd_vel', Twist, queue_size=10)
    rospy.sleep(1)
    rospy.loginfo("=== Iniciando secuencia ===")
    try:
        mover_recto(publisher, velocidad=0.2, distancia=1.0)
        rospy.sleep(1)
        girar_90_grados(publisher, velocidad_angular=0.5)
        rospy.loginfo("=== Secuencia completada ===")
    except rospy.ROSInterruptException:
        rospy.loginfo("Nodo interrumpido.")
```

5. Ejecutamos ROS

Mediante el siguiente comando que vemos, ejecutamos ROS, creando así el contenedor docker que contiene el sistema configurado.

```
p-full
PS C:\Users\jgortiz> docker run -it --name ros_practica -v C:\practica_ros:/practica_ros osrf/ros:noetic-deskto
p-full bash
root@dbd97b5dc587:/#
```

6. Copia del archivo Py al contenedor

Debido al uso de caracteres especiales en el nombre de distintas carpetas y archivos, creo manualmente la carpeta dentro del contenedor y copio el archivo python (ros.JulioGarcía.py) para poder ejecutarlo.

```
Run 'docker run --help' for more information
PS C:\Users\jgortiz> mkdir C:\ros_practica


    Directorio: C:\


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        04/03/2026     17:54                ros_practica


PS C:\Users\jgortiz> copy "C:\Users\jgortiz\Desktop\IA-BigData\MIA-Modelos de Inteligencia Artificial\Robótica
Control_Movimiento_Robot\ros.JulioGarcia.py" C:\ros_practica\
PS C:\Users\jgortiz> docker run -it --name ros_practica -v C:\ros_practica:/practica_ros osrf/ros:noetic-deskt
p-full bash
root@8f57f152df7f:/#
```

7. Actualización de TurtleBot3

Actualizamos los repositorios del sistema e instalamos los paquetes necesarios para trabajar.

```
root@8f57f152df7f:/# apt update
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Get:2 http://archive.ubuntu.com/ubuntu focal InRelease [265 kB]
```

```
root@8f57f152df7f:/# apt install ros-noetic-turtlebot3 ros-noetic-turtlebot3-simulations -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

```
root@8f57f152df7f:/# echo "export TURTLEBOT3_MODEL=burger" >> ~/.bashrc
root@8f57f152df7f:/# source /opt/ros/noetic/setup.bash
root@8f57f152df7f:/# rosversion -d
noetic
root@8f57f152df7f:/#
```

8. Ejecución del programa

Ejecutamos la secuencia y comprobamos que lo lance de forma correcta.

```
root@8f57f152df7f:/# roscore
... logging to /root/.ros/log/904d8bbe-17eb-11f1-a084-125338d0b412/roslaunch-8f57f152df7f-659.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://8f57f152df7f:44031/
ros_comm version 1.17.4
```

Una vez el proyecto esté lanzado y el entorno listo, copiamos los siguientes comandos en una nueva terminal, para ejecutar los movimientos programados mediante el archivo python.

source /opt/ros/noetic/setup.bash
cd /practica_ros
python3 ros.JulioGarcia.py

```
PS C:\Users\jgortiz> docker exec -it ros_practica bash
root@8f57f152df7f:/# source /opt/ros/noetic/setup.bash
root@8f57f152df7f:/# cd /practica_ros
root@8f57f152df7f:/practica_ros# python3 ros.JulioGarcia.py
[INFO] [1772645025.386498, 112.803000]: === Iniciando secuencia ===
[INFO] [1772645025.387964, 112.803000]: Moviendo recto 1.0 metros...
[INFO] [1772645030.892794, 118.305000]: Movimiento recto completado.
[INFO] [1772645031.893342, 119.305000]: Girando 90 grados...
[INFO] [1772645035.596971, 123.006000]: Giro completado.
[INFO] [1772645035.598813, 123.007000]: === Secuencia completada ===
root@8f57f152df7f:/practica_ros#
```

Aquí ya veríamos el resultado final que mostrará la correcta ejecución del archivo y los movimientos que ha realizado el robot.

9. Conclusión

En este proyecto hemos instalado ROS usando los contenedores de Docker, yo personalmente antes intente hacerlo directamente desde ubuntu pero debido a que estaba alojando el sistema en un USB y era una sesión de Ubuntu Live los procesos corrían todos en RAM y no se podían guardar los cambios por lo que daba error. Después también lo intente en una máquina virtual ya que supuse que sería más sencillo pero también me dio error debido a la ISO de ubuntu que tenía. Llegados a este punto decidí hacerlo mediante Docker como el resto de compañeros y con ayuda de mi pana Claude.
En general aprendimos a configurar ROS usando Docker para ejecutar un archivo python donde controlamos el movimiento del robot.